Architecture 1001:
x86-64 Assembly
JourneyToTheCenterOfMemcpy.c

Xeno Kovah – 2021
xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

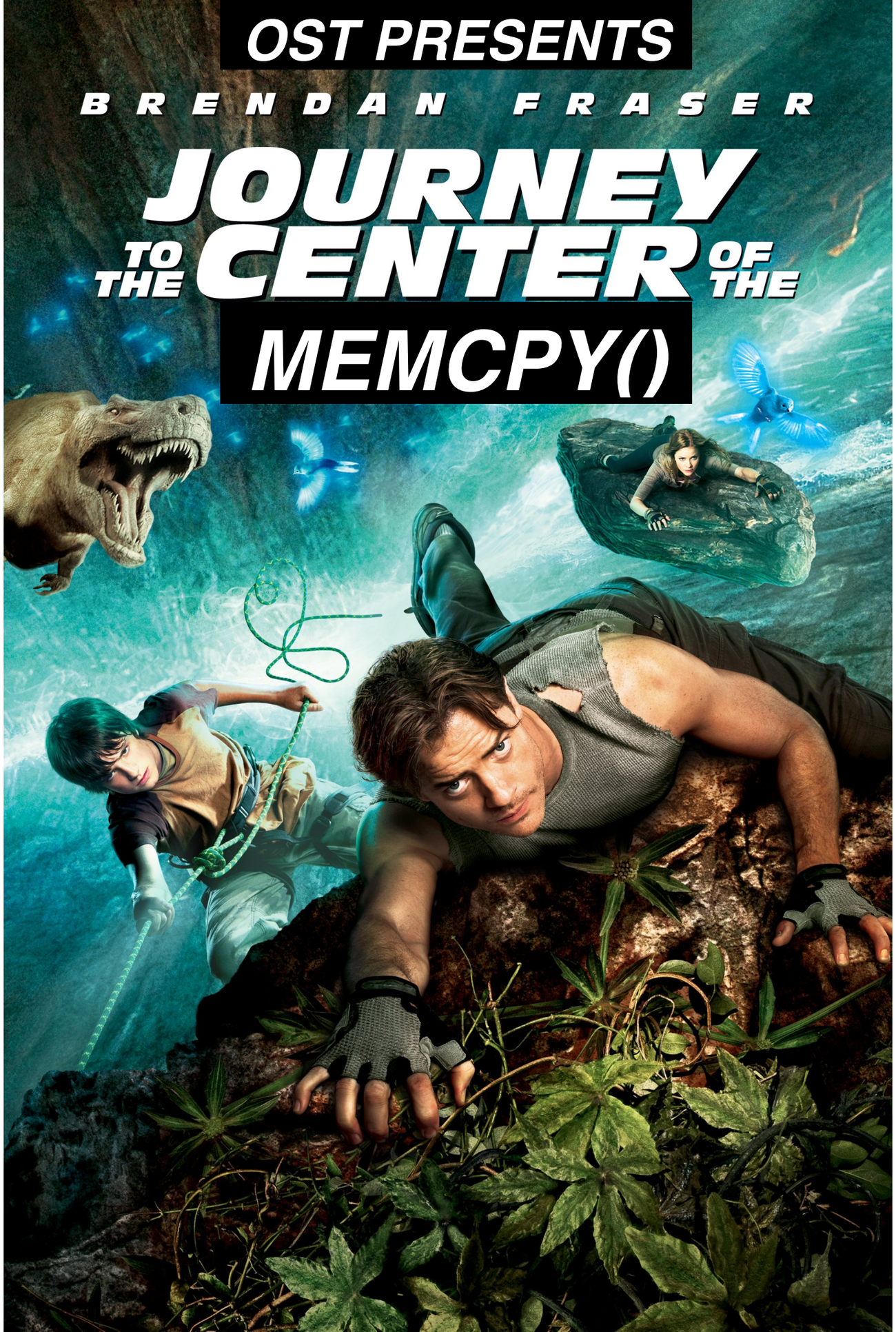
Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

OST PRESENTS

BRENDAN FRASER

JOURNEY TO THE CENTER OF THE MEMCPY()



JourneyToTheCenterOfMemcpy.c

```c
#include <stdio.h>

#pragma pack(1)
typedef struct mystruct{
    int var1;
    char var2[4];
} mystruct_t;
#pragma

#define uint64 unsigned long long
uint64 main(){
    mystruct_t a, b;
    a.var1 = 0xFF;
    memcpy(&b, &a, sizeof(mystruct_t));
    return 0xAce0fBa5e;
}
```

JourneyToTheCenterOfMemcpy.c

```c
#include <stdio.h>

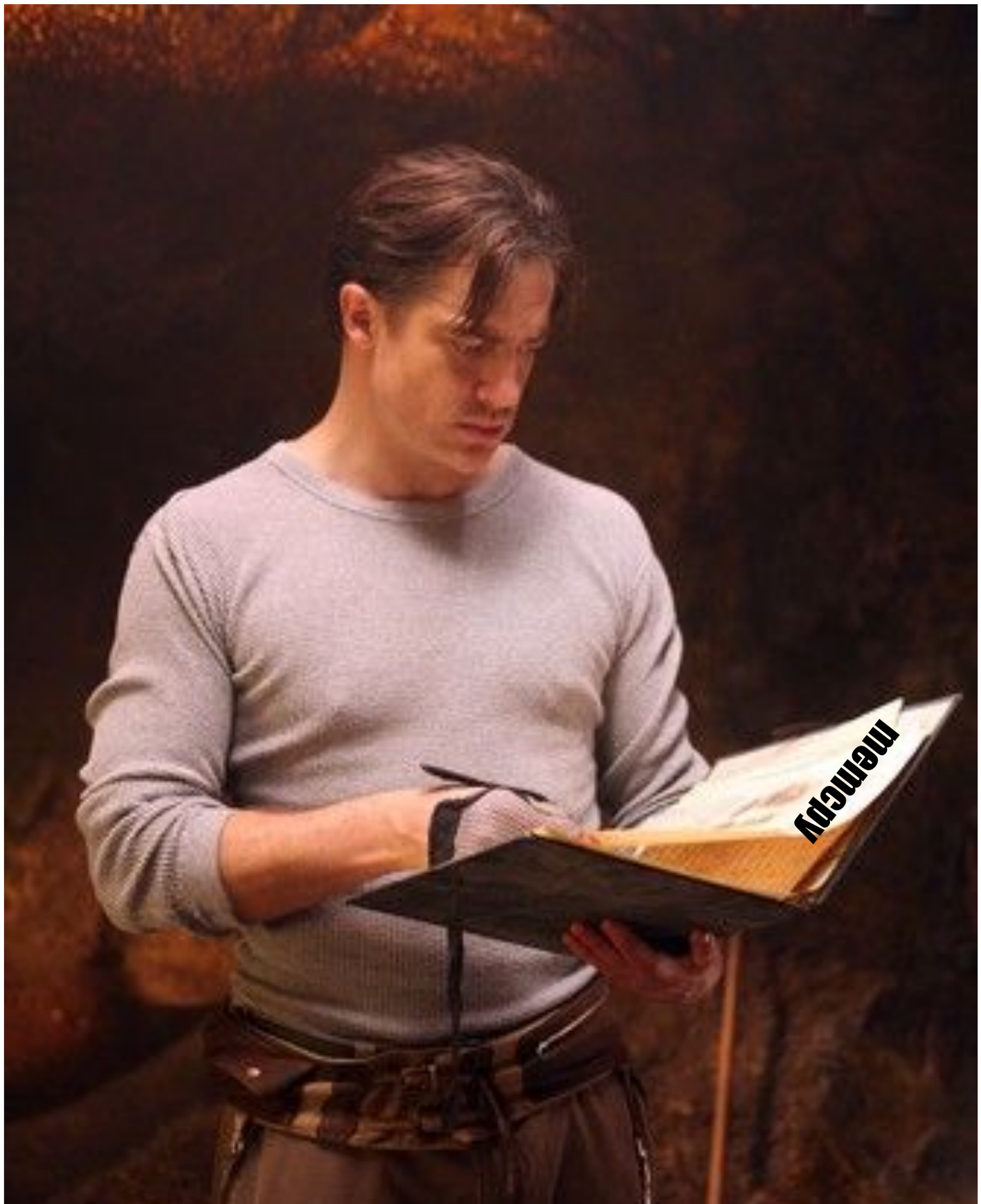
#pragma pack(1)
typedef struct mystruct{
    int var1;
    char var2[4];
} mystruct_t;
#pragma

#define uint64 unsigned long long
uint64 main(){
    mystruct_t a, b;
    a.var1 = 0xFF;
    memcpy(&b, &a, sizeof(mystruct_t));
    return 0xAce0fBa5e;
}
```

STEP INTO THE UNKNOWN!

```
sub   rsp,48h
mov   dword ptr [a],0FFh
mov   r8d,8
lea   rdx,[a]
lea   rcx,[b]
call memcpy (0140002E62h)
mov   rax,0ACE0FBA5Eh
add   rsp,48h
ret
```

Find REP MOVS!





STOP

&&

STEP THROUGH

THE MEMCPY ASSEMBLY

&&

FIND REP MOVS

&&

WRITE DOWN PSEUDOCODE

FOR THE PATH THAT LEADS TO IT



STOP

&&

STEP THROUGH

THE MEMCPY ASSEMBLY

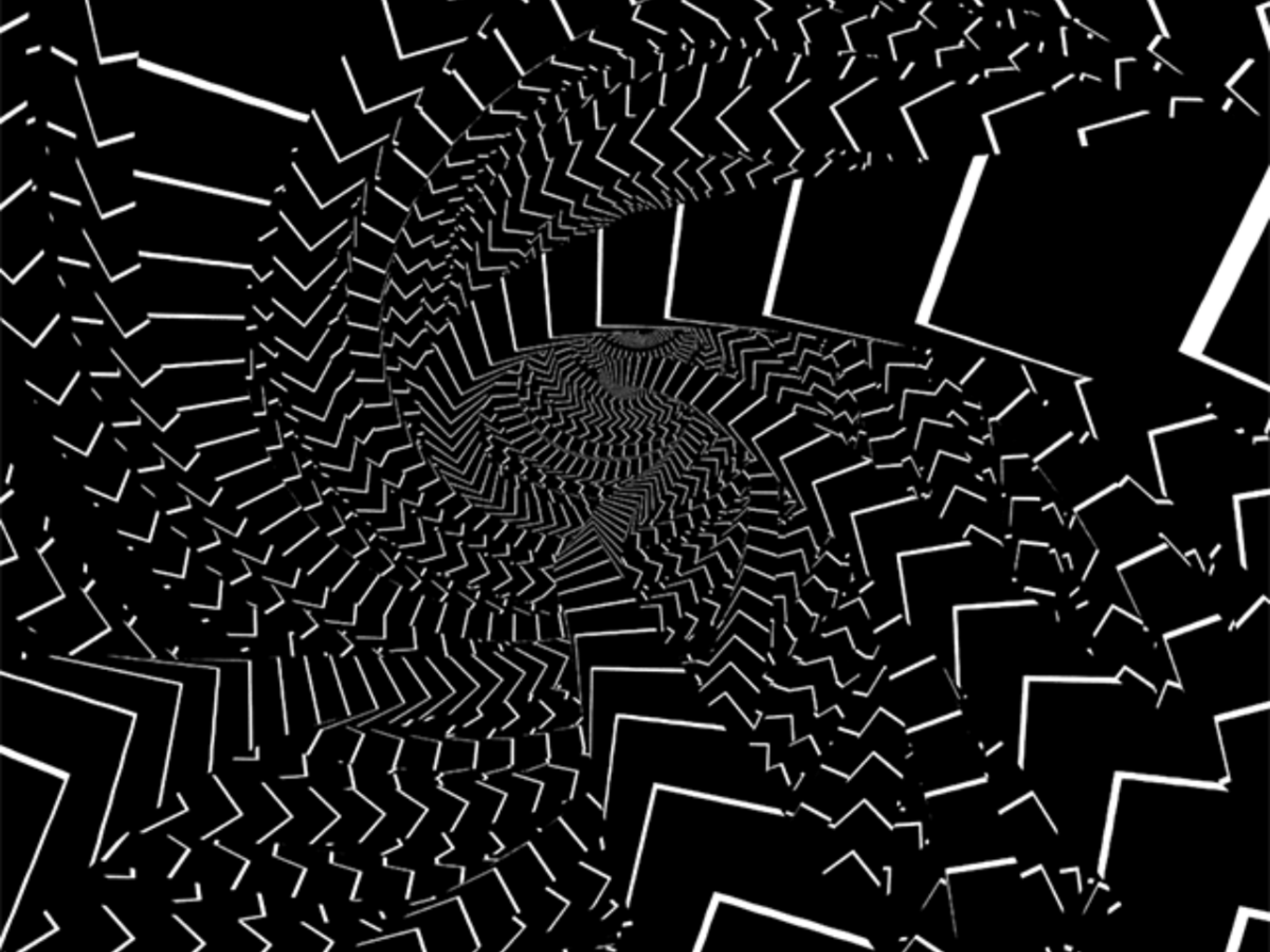
&&

FIND REP MOVS

&&

WRITE DOWN PSEUDOCODE

FOR THE PATH THAT LEADS TO IT



We'll Do It Live!

REP MOVS

Repeat **Mov**e Data String to **S**tring

- MOVS is one of number of instructions that can have the “rep” prefix added to it, which repeat a single instruction multiple times.
  - MOVS is actually its own instruction which can be called without the REP *instruction prefix*
- All rep operations use *cx register as a “counter” to determine how many times to loop through the instruction. Each time it executes, it decrements *cx. Once *cx == 0, it continues to the next instruction

Book page 101-102



REP MOVS 2

Repeat **Mov**e Data String to **S**tring

- REP MOVS stores 1, 2, 4, or 8 bytes at a time
- Either fill 1 byte from [si] to [di] with or fill 1/2/4/8 bytes at a time from [*si] to [*di]
- Moves the *di and *si registers forward 1/2/4/8 at a time, so that the repeated store operation is storing into consecutive locations.
- So there are 3 pieces which must happen before the actual rep movs occurs:
  - Set *si to the start source
  - Set *di to the start destination
  - Set *cx to the number of times to store

Book page 101-102



REP MOVS 3

Repeat **Mov**e Data String to **S**tring

- Note: Unlike MOV, MOVS can move memory to memory…but only between [*si] and [*di]
- A lot of people don't pay attention to the fact that it's REP *MOV***S**, not REP *MOV* (even though you may say it like "rep-move")

Book page 101-102

VS vs. GCC et al.
Caller vs. Callee-save Regs

VS Caller-saved	RAX		RCX	RDX				R8-11	
VS Callee-saved		RBX			RSI	RDI	RBP		R12-15
System V (GCC et al.) Caller-saved	RAX		RCX	RDX	RSI	RDI		R8-11	
System V (GCC et al.) Callee-saved		RBX					RBP		R12-15

High level pseudo-code approximation

(how interesting…it's like I went in *reverse* of the normal software *engineering* process…)

memcpy(void * dst, void * src, unsigned int len){

```
if(len <= 0x10)
    goto 00007FFB76491350 // 2x MOV
if(len <= 0x20)
    goto 00007FFB76491330 // AHHHH! MOVUPS! IDEKWTFBBQ!
else {
    if(src >= dst)
        goto 00007FFB76491314
    else { // if (src < len)
       if(dst < (src+dst)) // if (src < dst < (src + len)
          goto 00007FFB764915F0
    }
    if (len < 0x80){
        goto 00007FFB76491530 // AHHHH! MOVUPS! IDEKWTFBBQ!
    }
    else{
        if( ((*(char *)(7FFB764B43BCh)) & 0x2) == 0) // ?!
            goto 00007FFB76491480 // not taken ¯\_(ツ)_/¯
        else
            goto 00007FFB764912D0 // HERE BE REP MOVS!
    }
}
```

High level pseudo-code approximation

(how interesting…it's like I went in *reverse* of the normal software *engineering* process…)

```
memcpy(void * dst, void * src, unsigned int len){
if(len <= 0x10)
    goto 00007FFB76491350 // 2x MOV
if(len <= 0x20)
    goto 00007FFB76491330 // AHHHH! MOVUPS! IDEKWTFBBQ!
if (src < dst)
   if(dst < (src+len)) // Uh oh, overlap!
      goto 00007FFB764915F0 // not taken
}
if (len < 0x80){
    goto 00007FFB76491530 // AHHHH! MOVUPS! IDEKWTFBBQ!
}
else{
    if( ((*(char *)(7FFB764B43BCh)) & 0x2) == 0) // ?!
        goto 00007FFB76491480 // not taken ¯\_(ツ)_/¯
    else
       goto 00007FFB764912D0 // HERE BE REP MOVS!
}
...
}
```

This Class

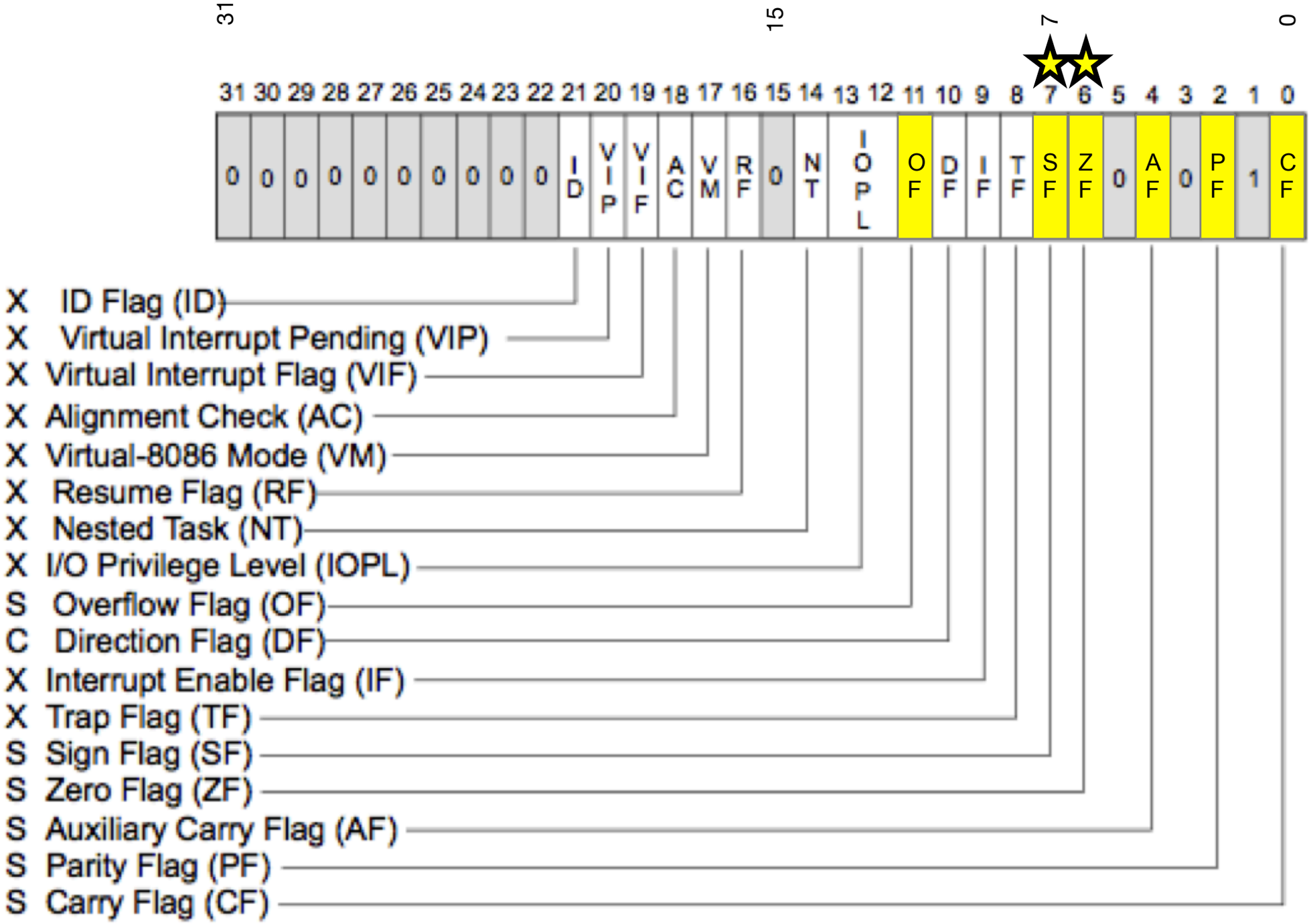


Figure 3-8. EFLAGS Register

This Class

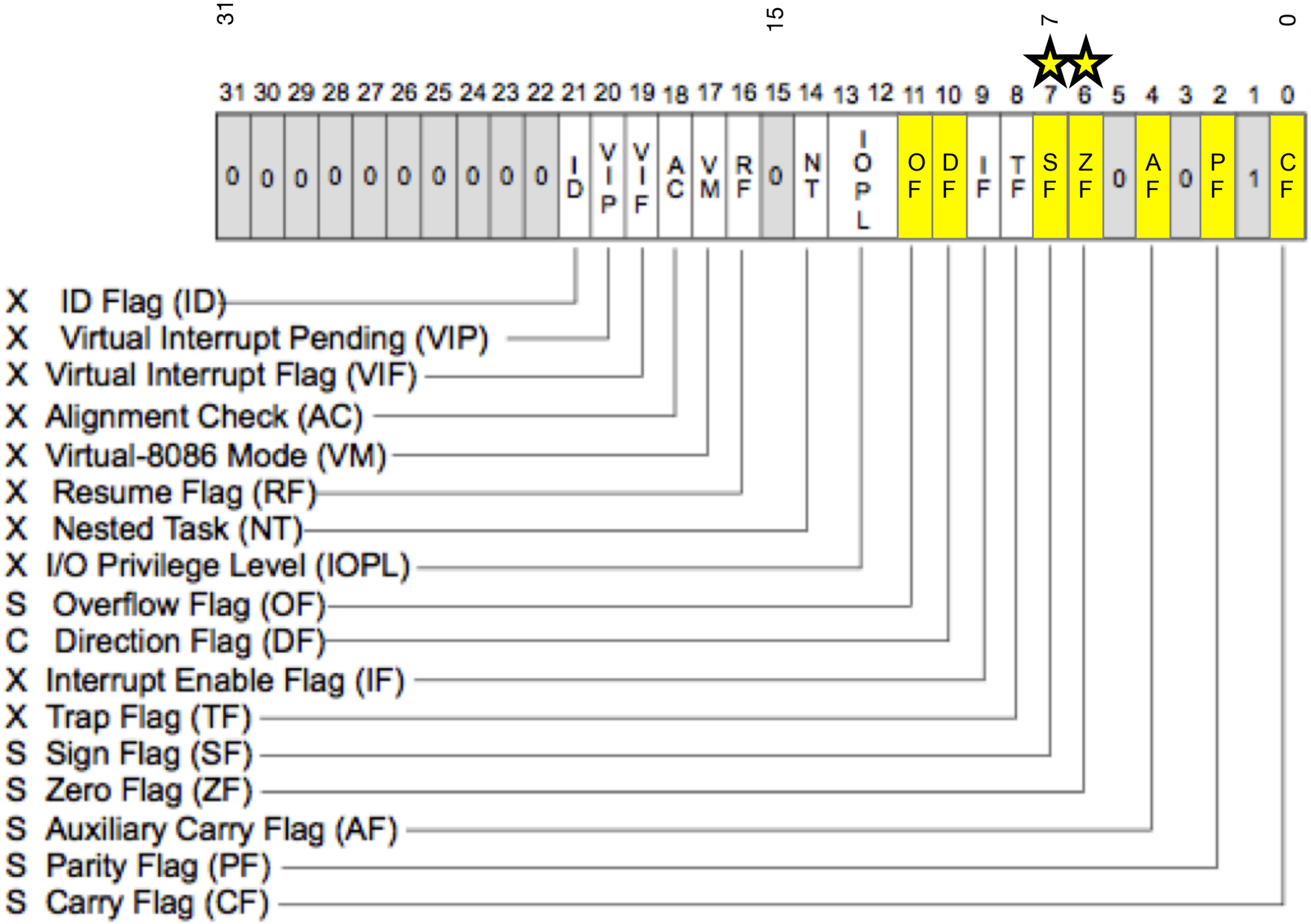


Figure 3-8. EFLAGS Register

This Class

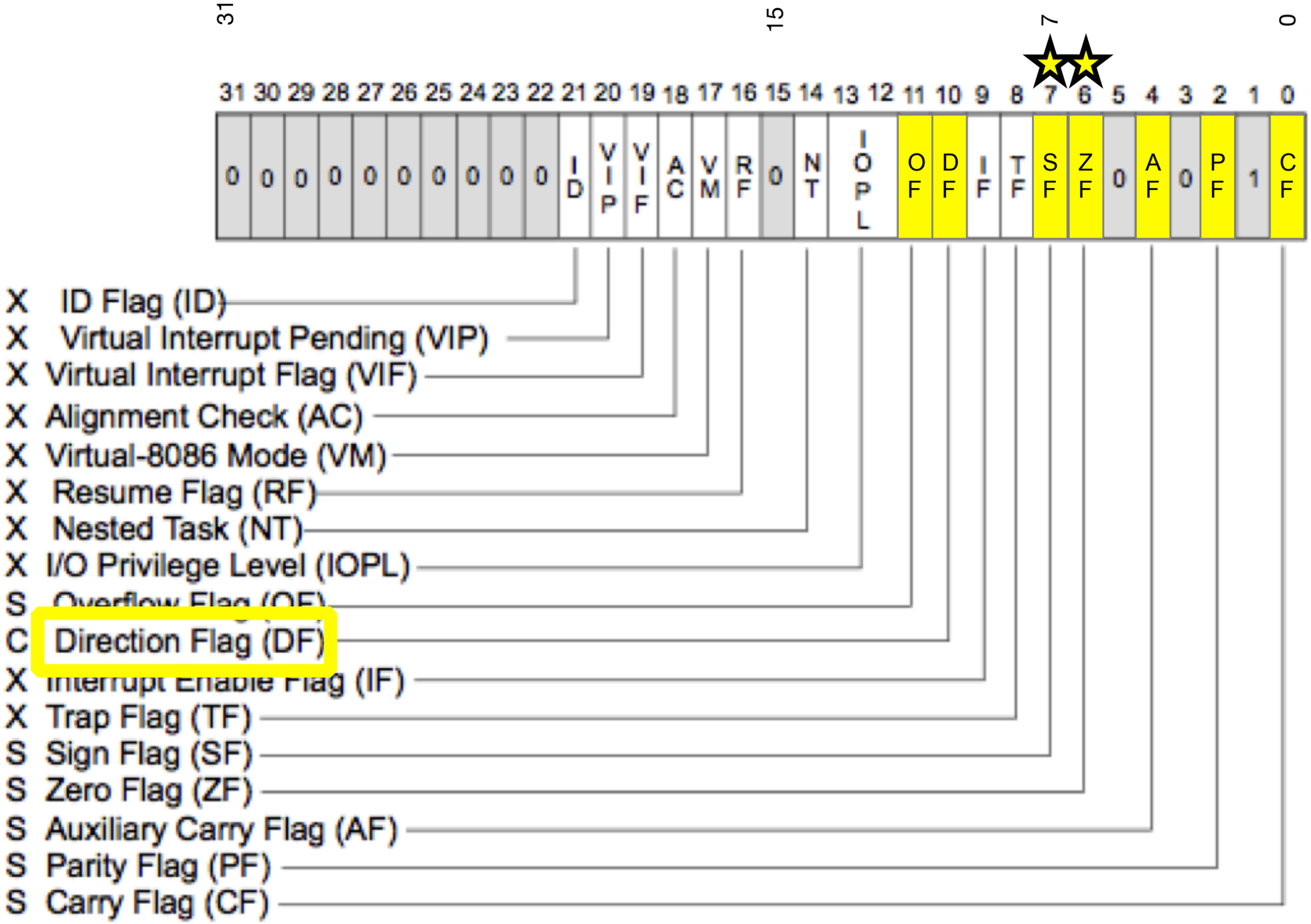


Figure 3-8. EFLAGS Register

Instructions we now know (30)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- IMUL
- MOVZX/MOVSX
- LEA
- JMP/Jcc (family)
- CMP/TEST
- AND/OR/XOR/NOT
- INC/DEC
- SHR/SHL/SAR/SAL
- DIV/IDIV
- REP STOS
- **REP MOVS**

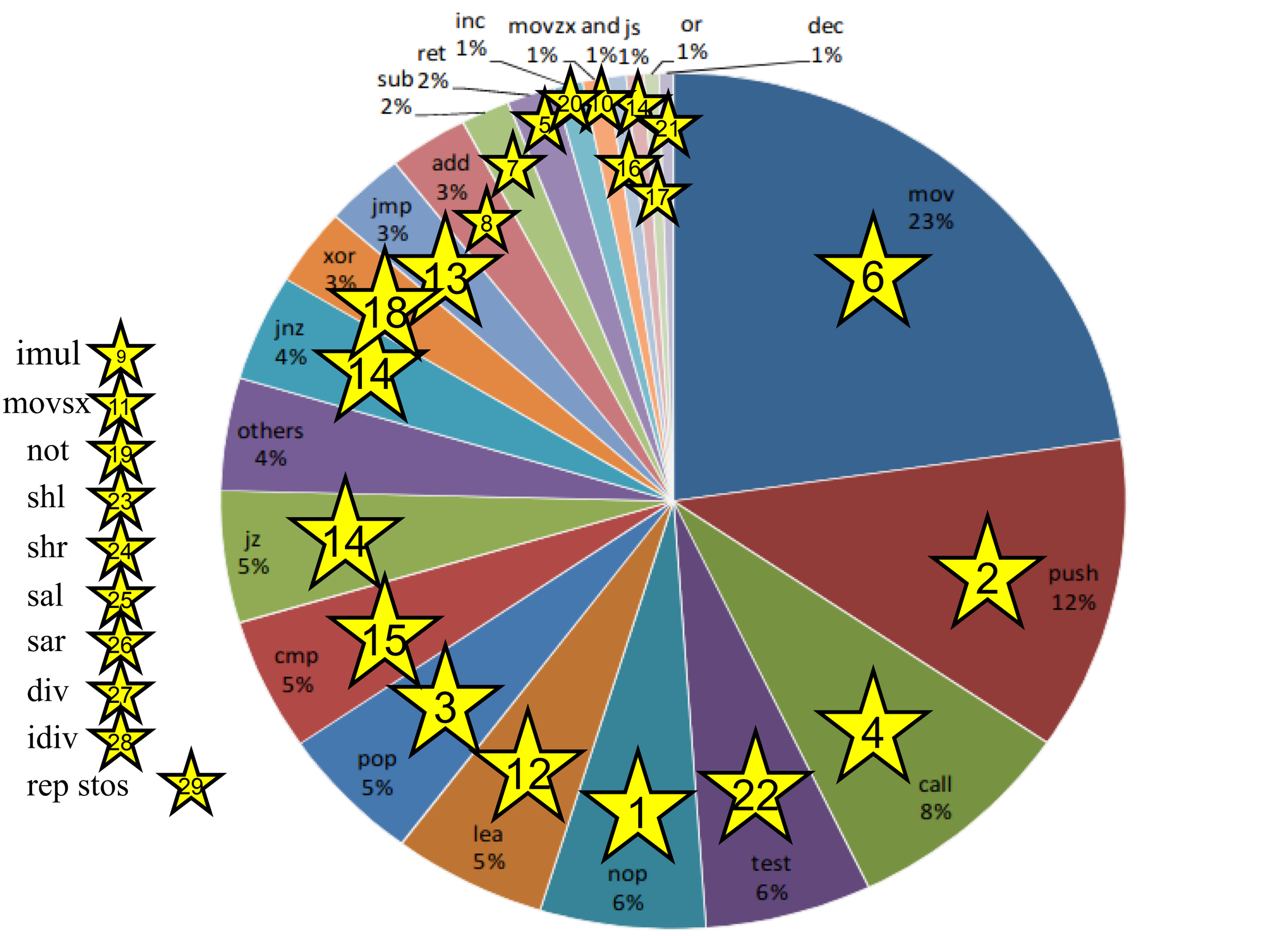


Fig. 2 Web browsers' instruction frequencies

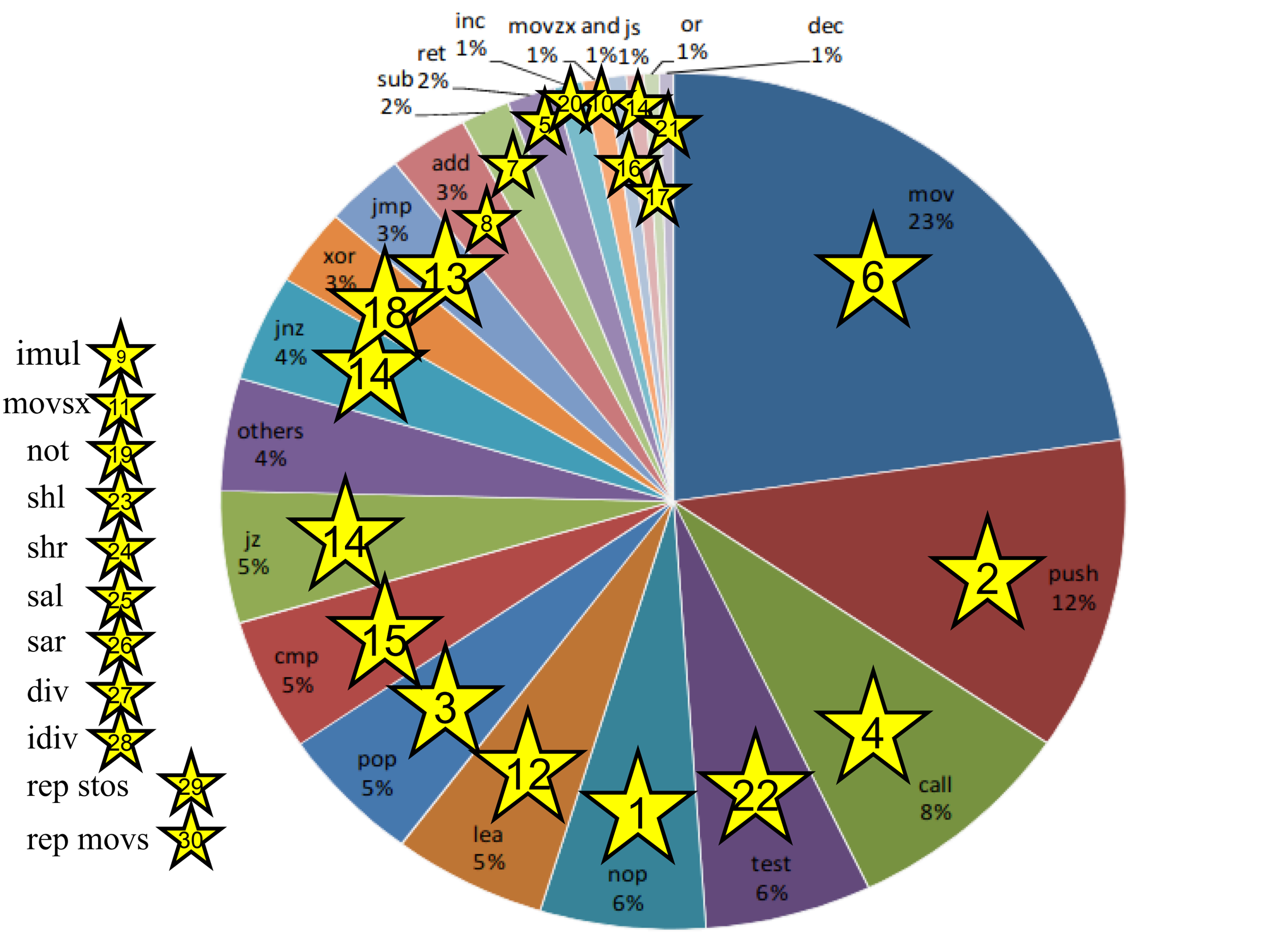


Fig. 2 Web browsers' instruction frequencies